

Hermes Agent for Beginners

Sample chapter — the curator

Harri Pehkonen

2026

Contents

The curator: maintenance for a library that grows itself	1
What you will be able to do	1
The idea	1
Do this	2
Verify it	5
What broke for me	5
Pitfalls	6
Going all in, honestly	6
Where to go next	7
Key ideas	8
Index	9

The curator: maintenance for a library that grows itself

DEFINITIVE — The curator only ever touches skills it has been given

Before you change anything: the curator manages skills whose provenance says an agent created them. Skills you wrote, and skills that ship with Hermes, are out of its reach until you hand one over with `hermes curator adopt`. It also never deletes — the worst outcome is an archive you can restore.

What you will be able to do

Run the curator deliberately, let it run unattended, and undo anything it does — including a single change it made three weeks ago.

The idea

Here is the problem it solves. Hermes creates skills by itself: after a session where something novel got worked out, a background pass writes the procedure down as a skill. That is the feature that makes the agent improve, and it is also the feature that guarantees your library grows whether or not you are paying attention. Nothing about that growth is visible in a chat window.

The curator is the drain for that tap. It is a maintenance pass with **two phases**, and the difference between them decides how much you need to care:

Phase	Cost	What it does
Automatic transitions	free, always on	Walks each managed skill through active → stale → archived based on how long it has gone unused

Phase	Cost	What it does
Consolidation	50-100 model calls per pass, opt-in	A model reads the managed skills and decides, per skill: keep, patch, merge overlapping ones into an umbrella, or archive

The first phase is bookkeeping and you can ignore it forever. The second phase is a judgement call made by a model about the files that steer every future session — so it is off by default, and it is the only part of the curator that needs a habit from you.

What it is allowed to touch

The gate is provenance, stored per skill, and it divides your library into two groups:

Group	Examples	Curator may
Agent-created	skills written by a self-improvement pass	prune, archive, patch, merge
Everything else	skills you wrote or edited by hand; skills shipped with Hermes; anything installed from the skills hub	nothing, unless you adopt it (<code>prune_builtins</code> is the one opt-in that lets old bundled skills be archived; hub skills are always exempt)

Two more fences sit outside that table. **Pinned** skills are skipped by every automatic transition — and, worth knowing before you use it as an “important” flag, pinning also blocks the agent’s own skill-editing tool, so it means “nobody touches this”. And any skill **referenced by a scheduled job**, even a paused one, is skipped for the same reason a pinned skill is: a slow schedule must not be able to archive a skill out from under a job that expects it.

Do this

1. Look before you touch

```
$ hermes curator status
curator: ENABLED
  runs:          0
```

```

last run:      never
last summary:  (none)
interval:     every 7d
stale after:   30d unused
archive after: 90d unused
consolidate:   off (prune-only; LLM merge pass opt-in)

```

```
curator-managed skills: 112 total (agent-created=31 bundled=81)
```

```

active      112
stale       0
archived    0

```

```
unmanaged (no provenance marker): 60 total
```

```

pre-dates marker    31
foreground-created   29

```

```
never auto-staled or archived – `hermes curator adopt <name>` hands one over
```

Four numbers matter here: how many skills it manages, how many are unmanaged, how many are already stale or archived, and what the thresholds are. The most active / least recently active lists at the end of the command are what tell you whether a threshold is about to fire.

`hermes curator usage` is the same telemetry for every skill regardless of provenance, and `hermes curator list-unmanaged` is the adoption candidate list — it is also the command that answers “why can’t the agent edit this skill?”.

2. Ask what it would do, in a run that cannot change anything

```
$ hermes curator run --dry-run
```

This produces the full review report and mutates nothing. Do this before the first real run, and after any change to your thresholds.

3. Fence off what must not move

```

$ hermes curator pin local-quality-gates
curator: pinned 'local-quality-gates' (will bypass auto-transitions)

```

```
$ hermes curator unpin local-quality-gates
```

Pin sparingly, and remember the bidirectional fence: pinned also means the agent cannot patch it.

4. Hand over what you want managed

```
$ hermes curator list-unmanaged
$ hermes curator adopt cpp-build-toolchain
```

Adoption is a declaration by you, not a discovery by the tool, which is why it is a command rather than a default: it is the moment a skill you are responsible for becomes one the curator may rewrite. Adopt deliberately, in small batches.

5. Let it run

```
$ hermes config get curator.enabled          # true
$ hermes config get curator.interval_hours   # 168
$ hermes config get curator.min_idle_hours   # 2
```

This is the set-it-and-forget-it part, and it is genuinely hands-off:

- The curator is **not a scheduled job**. It is an inactivity check, run when a CLI session starts, when the gateway does housekeeping, and by the desktop or hermes serve maintenance timer. It fires only when both `interval_hours` has passed **and** the profile has been idle `min_idle_hours` — so it works while you are away from the keyboard and never competes with a live conversation.
- **The first pass is deferred on purpose**. On a new install the first observation seeds the clock and the first real pass waits one full interval. A long quiet period after installing is the design, not a bug.
- Tune the thresholds in `config.yaml` (settings belong here, never in `.env`):

```
curator:
  enabled: true
  interval_hours: 168
  min_idle_hours: 2
  stale_after_days: 30
  archive_after_days: 90
  consolidate: false
  prune_builtins: false
```

- To make the review pass cheap, remember it is an ordinary auxiliary model slot: `hermes model` has a **Curator** entry, and `auto` means “use my main chat model”. Point it at something small and the reviews cost pocket change.

6. The optional heavy pass

```
$ hermes curator run --consolidate
```

Expect it to take a while — it is dozens of model calls. Read it afterwards, not during:

```
$ hermes curator ledger | tail -30
```

7. Rescue

```
$ hermes curator ledger                # every change, every actor, with ids
$ hermes curator rollback 1a2b3c4d5e6f # undo ONE change by entry id
$ hermes curator list-archived         # what left the index
$ hermes curator restore <skill>       # bring one back
$ hermes curator archive <skill>       # put one away by hand
$ hermes curator prune --days 120     # bulk-archive idle skills
$ hermes curator backup                # snapshot the whole library
$ hermes curator purge --yes           # delete archives for good (manual only)
```

purge is the only destructive verb in the list, it is never automatic, and it is gated by `curator.archive_ttl_days`.

Verify it

- **The ledger is the record.** One JSONL row per mutation, with the actor, the skill, the changed paths, and an entry id you can roll back. Count rows before and after.
- **Archives are files.** `ls skills/.archive/` shows what left the index; the skill is still on disk and restorable.
- **Backups are blobs.** Per-mutation file contents live under `~/hermes/.curator_backups/blobs` (content-addressed), which is what makes single-mutation rollback possible.
- **Do not use status to prove a run happened.** See the incident below.

What broke for me

The ledger lies about who did what. Every skill mutation appends to `skills/.curator_ledger.js` and the actor is derived: an explicit override wins (the CLI says `user`, the curator's own walk says `curator`), and otherwise the background-review provenance signal is recorded as `curator`. In practice that means rows tagged `curator` are mostly the post-turn review fork — on this install, 533 such rows accumulated while `status` still reported `runs: 0`, and the number counts work the curator never did.

And then it inverted. After a completed manual `hermes curator run --consolidate`, the same `status` command still reported `runs: 0 / last run: never`. So the `status` line is not evidence that a run happened either. The ledger is the only trustworthy record, and the actor column inside it needs interpreting. Cross-check both, and trust neither alone.

The documentation and the install disagree. The published defaults for `stale_after_days` and `archive_after_days` were 14 and 30; the resolved values on a real install were 30 and 90. Nothing was broken — the console simply beats the docs page. Read `hermes config get curator.<key>` before reasoning about what a

threshold will do.

The run wrote a skill while we were talking about it. Watching the ledger during a session showed a new skill appear, on the very topic under discussion, minutes after the conversation about it. That is the growth engine working in real time — and the clearest illustration of why a drain matters: the tap is automatic, so the removal has to be too.

There was no whole-library snapshot to fall back on. The header comment in `curator_backup` says a tarball is taken before any mutating pass; on this install the backup directory contained only per-mutation blobs, and no `skills.tar.gz` at all. The rollback you actually have is one file at a time.

Pitfalls

- **Consolidation is a model's opinion.** A merge can flatten nuance, drop a pitfall, or combine two skills you think of separately — and when it merges, it also rewrites references to those skills from your scheduled jobs. Review the ledger diff after a consolidate run, or accept drift.
- **prune_builtins: true makes skills disappear.** A bundled skill that goes unused past the archive window leaves the index, which looks exactly like a broken installation. It is recoverable, but you will not know to recover it unless you are reading the ledger.
- **Adoption is a door, not a status.** Bulk-adopting every unmanaged skill hands the curator your hand-written work in one move. Adopt the ones you want maintained.
- **Pin is bidirectional.** It stops the agent from patching the skill as well as the curator from archiving it.
- **Nothing expires on its own.** There is no TTL, no ephemeral flag, no auto-cleanup in the skill loader. If you want a skill gone, something you own has to remove it.

Going all in, honestly

The design is safe against loss: it never deletes, archives are restorable, and every mutation is individually reversible. It is not safe against drift, and drift is invisible unless something looks. So the honest version of set-and-forget is three commands of insurance and one passive check:

1. `hermes curator backup` once, so you have a whole-library rollback point.
2. `pin` the handful of skills you would hate to see rewritten.
3. Leave `consolidate: false` unless you want umbrella merges, and run it one-off when you do.

4. Add a weekly job that reports only when the curator changed something — new archives, merges, or ledger rows since the last run — and stays silent when nothing moved. That is the checking, without the reading.

Where to go next

The curator keeps the procedure library honest. The next chapter is about the other half of maintenance: the per-turn budget the skill index spends, and the monthly checklist that keeps memory, skills and backups in shape.

Key ideas

- The curator only ever touches skills it has been given

Index

Consolidation **The idea**

curator **What you will be able to do**

provenance **What it is allowed to touch**

skill **The idea**